UTF-8 Generalized Glyph

Any future glyph (including animated glyphs) that are uniquely distinguishable from each other within reason for representing semantic meaning in any future discovered or invented language, can be shown using this last and final update to the UTF-8 framework.

The de facto maximum memory used per glyph is 261744 bytes, but, once a glyph is used once, it is saved, and the next text using it only needs a minimum of 15 bytes using this framework to make a "Reference Character", or somewhere between 43 and 100 bytes to have a "Reference Character" that has considerable modifications using the original glyph so saved on that computer (by moving the camera around within the space and time in which the curves of the original glyph exist, practically meaning that the glyph can be inverted, turned upside down, shrunk, cropped, or roughly made to appear animated by having the camera move among various curves in a path defined by polynomial parametric equations).

Underlying basis of this framework:

First byte (declaration of using this system):	11111110
Second byte:	10xxxxxx
Third byte:	10xxxxxx
Fourth byte:	10xxxxxx
Fifth byte:	10xxxxxx
Sixth byte:	10xxxxxx
Seventh byte:	10xxxxxx
Eighth byte:	10xxxxxx
Ninth byte:	10xxxxxx
Tenth byte:	10xxxxxx

Eleventh byte:	10xxxxxx
Twelfth byte:	10xxxxxx
Thirteenth byte:	10xxxxxx
Fourteenth byte:	10xxxxxx
Fifteenth byte:	10xxxxxx

The 66 x's of the bytes from the Second byte to the Twelfth byte is the "Name" of this glyph, which must be registered in a hashmap of a computer that saves the glyph, thus allowing a "Reference Character" later to directly name a glyph and have it represented instead of needing to include another 65535 bytes for the character.
The 18 x's within bytes from the Thirteenth byte to the Fifteenth byte must be interpreted as an 18 bit unsigned integer which states the length (in bytes) of this glyph.

If the length is less than 101, then this character shall be interpreted as a "Reference Character", where the curves are to be drawn from computer memory of the most recent version of the character with the same initial 12 bytes.
If a Reference Character is less than 43 bytes long, then use the camera position stated in that most recent version of the character where the length was greater than 101 bytes.
If a Reference Character has a length between 43 and 46 bytes, then use the camera position stated within the below camera information (a static camera).
If a Reference Character has a length between 44 and 100, then use the camera position and movement stated within the below camera information (assume values of 0 for blank spots that couldn't be properly filled).
For all Reference Characters, the actually drawn curves which are to be observed must be the ones created by following the curve equations in the saved glyph.

Camera information:
Sixteenth byte: 10xxxxxx
Seventeenth byte: 10xxxxxx
Eighteenth byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the x-coordinate of the camera initially.)
Nineteenth byte: 10xxxxxx
Twentieth byte: 10xxxxxx
Twenty-first byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the y-coordinate of the camera initially.)
Twenty-second byte: 10xxxxxx
Twenty-third byte: 10xxxxxx
Twenty-fourth byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the z-coordinate of the camera initially.)
Twenty-fifth byte: 10xxxxxx
Twenty-sixth byte: 10xxxxxx
Twenty-seventh byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the x-coordinate of the camera's top leftmost location its view could theoretically reach if no curves were in the way (would be the top left in the rendered image) initially.)
Twenty-eighth byte: 10xxxxxx
Twenty-ninth byte: 10xxxxxx
Thirtieth byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the x-coordinate of the camera's top

leftmost location its view could theoretically reach if no curves were in the way (would be the top left in the rendered image) initially.)
Thirty-first byte: 10xxxxxx
Thirty-second byte: 10xxxxxx
Thirty-third byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the x-coordinate of the camera's top leftmost location its view could theoretically reach if no curves were in the way (would be the top left in the rendered image) initially.)
Thirty-fourth byte: 10xxxxxx
Thirty-fifth byte:10xxxxxx
Thirty-sixth byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the x-coordinate of the camera's top leftmost location its view could theoretically reach if no curves were in the way (would be the top left in the rendered image) initially.)
Thirty-seventh byte: 10xxxxxx
Thirty-eighth byte: 10xxxxxx
Thirty-ninth byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the x-coordinate of the camera's top leftmost location its view could theoretically reach if no curves were in the way (would be the top left in the rendered image) initially.)
Fourtieth byte: 10xxxxxx
Fourty-first byte: 10xxxxxx
Fourty-second byte: 10xxxxxx (these 6 x's plus the 12 immediately prior x's are interpreted as the value (unsigned 18 bit integer - 131071.0)/1310.72, this becomes the x-coordinate of the camera's top leftmost location its view could theoretically reach if no curves were in the way (would be the top left in the rendered image) initially.)
Fourty-third byte: 1000000x (x=0 if the camera's position remains static for the duration of the document's existence on screen, x=1

means the camera moves according to the following equations with respect to w (number of seconds since the page loaded), that the camera will be at those specific coordinates at that time.)
Fourty-fourth byte: 10xxxxxx
Fourty-fifth byte: 10xxxxxx
Fourty-sixth byte: 10xxxxxx (Interpret like an 18 bit unsigned integer for when the model is supposed to repeat (measured in seconds from when the page was loaded, at which point the number of seconds from when the page was loaded is set to 0 internally).)

The remaining bytes are treated as being used in groups of 3 where the ending 6 bits of each byte are concatenated together as an unsigned 18 bit integer, then subtract 131071.0 from it, and divide that difference by 1310.72 to get a floating point number to put in the blank spot.

Equation of Camera's motion (where w is seconds from when the page loaded):
x(w)= max(_, min(_, _ + _*w + _*w*w + _*w*w*w));
y(w)= max(_, min(_, _ + _*w + _*w*w + _*w*w*w));
z(w)= max(_, min(_, _ + _*w + _*w*w + _*w*w*w));

This character is meant to always take up exactly 65535 bytes (2 to the power of 16 minus one) (except where the artist chooses to make the glyph representation smaller or bigger using the length attribute (limited to a range from 0 (should be treated by the computer as an error) to 262143 bytes)).

We use 8 parametric equations for each curve within the figure, where the data encoded in the bits of the glyph in the PDF or

webpage (or wherever UTF-8 characters get used) represent filling the "_" spaces in each of the following equations:

Curve i:
The color of this curve is _ (4 bytes that go through operations as: The x's in 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx get translitterated into "10rrrrrr 10rrgggg 10ggggbb 10bbbbbb". The red color value thus goes from 0 to 255 using the bits "rrrrrrrr", the green color value thus goes from 0 to 255 using the bits "gggggggg", and the blue color value goes from 0 to 255 using the bits "bbbbbbbb".)
Parameter "t" is set to the floating point values from _ to _
The position of the middle of the curve in the x-coordinate when parameter t is at a relevant value above:
x(t)= _ + _*t + _*t*t + _*t*t*t + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t);
The width of the curve along the x-coordinates when parameter t is at a relevant value above:
width_x(t)= _ + _*t + _*t*t + _*t*t*t + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t);
The position of the middle of the curve in the y-coordinate when parameter t is at a relevant value above:
y(t)= _ + _*t + _*t*t + _*t*t*t + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t);
The width of the curve along the y-coordinates when parameter t is at a relevant value above:
width_y(t)= _ + _*t + _*t*t + _*t*t*t + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t);

The position of the middle of the curve in the z-coordinate when parameter t is at a relevant value above:
z(t)= _ + _*t + _*t*t + _*t*t*t + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t);
The width of the curve along the z-coordinates when parameter t is at a relevant value above:
width_z(t)= _ + _*t + _*t*t + _*t*t*t + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t) + _*sin(_ + _*t);
The existence of the curve at the position (x(t), y(t), z(t)) at the current time (in seconds) is true if (w(t)-width_w(t) < current_time_in_seconds AND current_time_in_seconds < w(t) +width_w(t)) is true:
w(t)= _ + _*t + _*t*t + _*t*t*t + _*t*t*t*t + _*t*t*t*t*t + _*t*t*t*t*t*t + _*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t*t*t*t*t;
The width of the existence of the part of the curve (measured in seconds both before and after the w-time above) at point (x(t), y(t), z(t)):
width_w(t)= _ + _*t + _*t*t + _*t*t*t + _*t*t*t*t + _*t*t*t*t*t + _*t*t*t*t*t*t + _*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t*t*t*t + _*t*t*t*t*t*t*t*t*t*t*t*t;

Bytes used up over 123 curves:	65436
Bytes used up per curve:	532
Color:	4
t minimum and maximum:	6
x(t) constants and coefficients:	75
width_x(t) constants and coefficients:	75
y(t) constants and coefficients:	75
width_y(t) constants and coefficients:	75

z(t) constants and coefficients:	75
width_z(t) constants and coefficients:	75
w(t) constants and coefficients:	36
width_w(t) constants and coefficients:	36

For an incomplete curve (where the length of the glyph is set smaller than what would fit a full 532 byte curve), assume every remaining empty spot "_" in the curve is set to 0.